

Introduction to Computer Organisation and Architecture UCCD1133



Chapter 6

Processor, Memory System and Instruction Execution

Disclaimer



- This slide may contain copyrighted material of which has not been specifically authorized by the copyright owner. The use of copyrighted materials are solely for educational purpose. If you wish to use this copyrighted material for other purposes, you must first obtain permission from the original copyright owner.

CH 6.2:

Processors: Single-cycle, Multi-cycle and Pipeline

Microarchitecture:

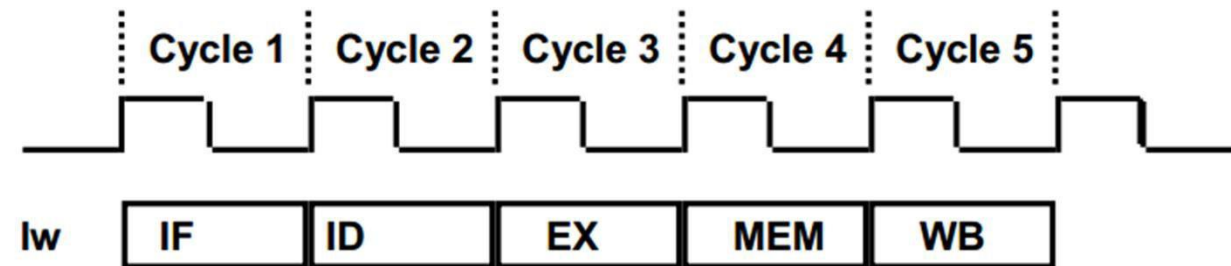
- Single-cycle Processor
- Multicycle Processor
- Pipelined Processor

Pipelined Processor

- Pipelining is an implementation technique in which **multiple instructions are overlapped** in execution.
- Today, pipelining is nearly universal.
- MIPS Instruction Set is designed for pipelining.
 - All instructions are of the same length – 32 bits
 - Easier to fetch and decode in one cycle

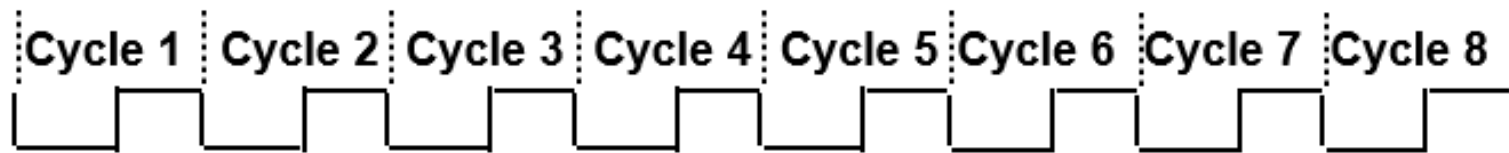
Pipelined Processor

- Recall that an instruction can be executed in stages (with each stage partially-completed). This is seen previously in single-cycle and multi-cycle processors.
 - IF**: Instruction Fetch, Increment PC
 - ID**: Instruction Decode, Read Registers
 - EX**: Execution (ALU)
- Load/Store: Calculate Address
- Others: Perform Operation
 - MEM**:
- Load: Read Data from Memory
- Store: Write Data to Memory
 - WB**: Write the result (Data) Back to Register file
- The term “stages” implies datapath resources at each stage



Pipelined Processor

- Pipelined processor start the **next** instruction before the current one has completed
 - improves throughput** - total amount of work done in a given time
 - instruction **latency** (execution time, delay time, response time - time from the start of an instruction to its completion) **is not reduced**



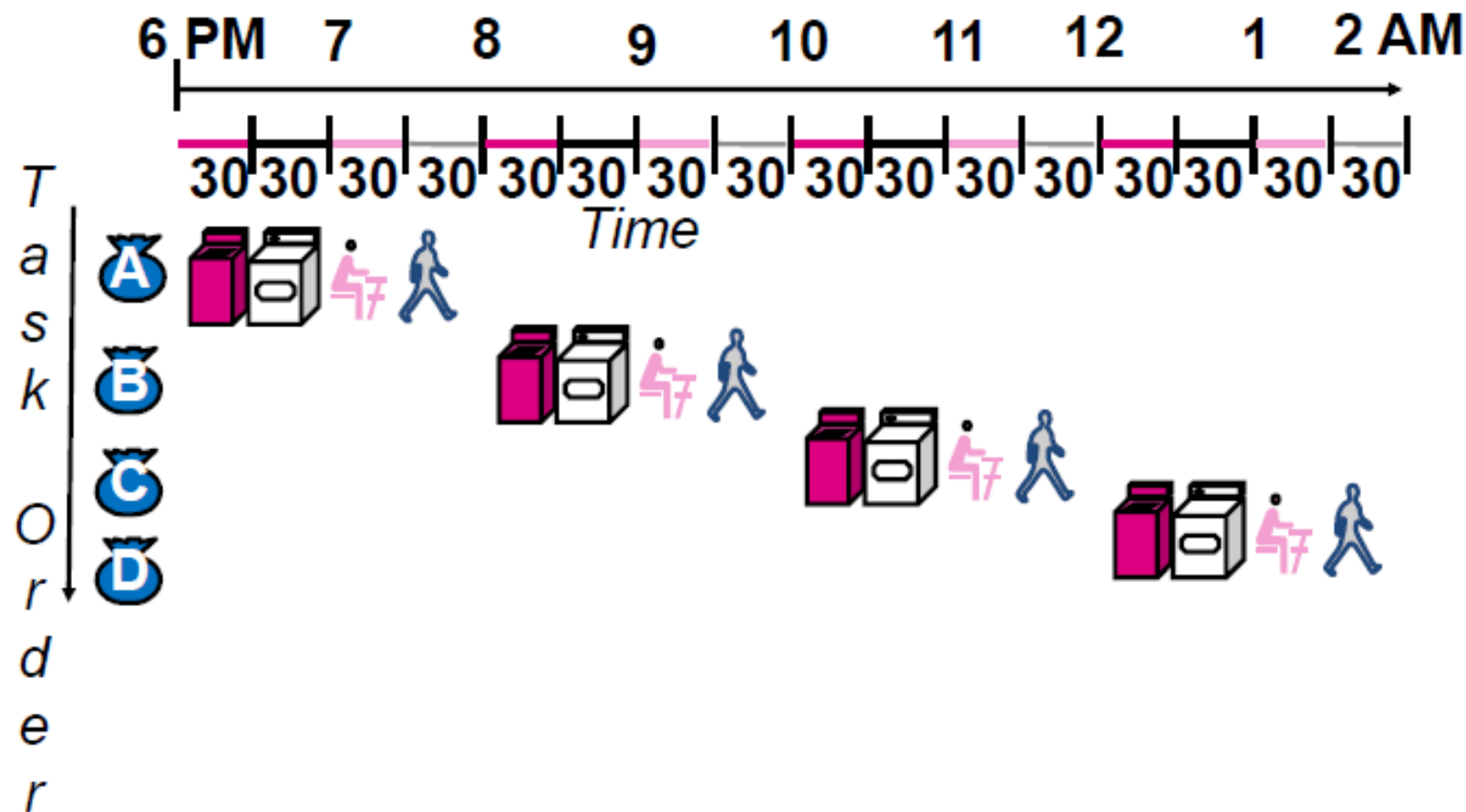
clock cycle (pipeline stage time) is limited by the slowest stage
for some instructions, some stages are wasted cycles

Pipelined Analogy: Doing Laundry

- Andy, Benny, Charles, and Denny each have one load of clothes to wash, dry, fold, and put away
 - Washer takes 30 minutes
 - Dryer takes 30 minutes
 - “Folder” takes 30 minutes
 - “Stasher” takes 30 minutes to put clothes into drawers

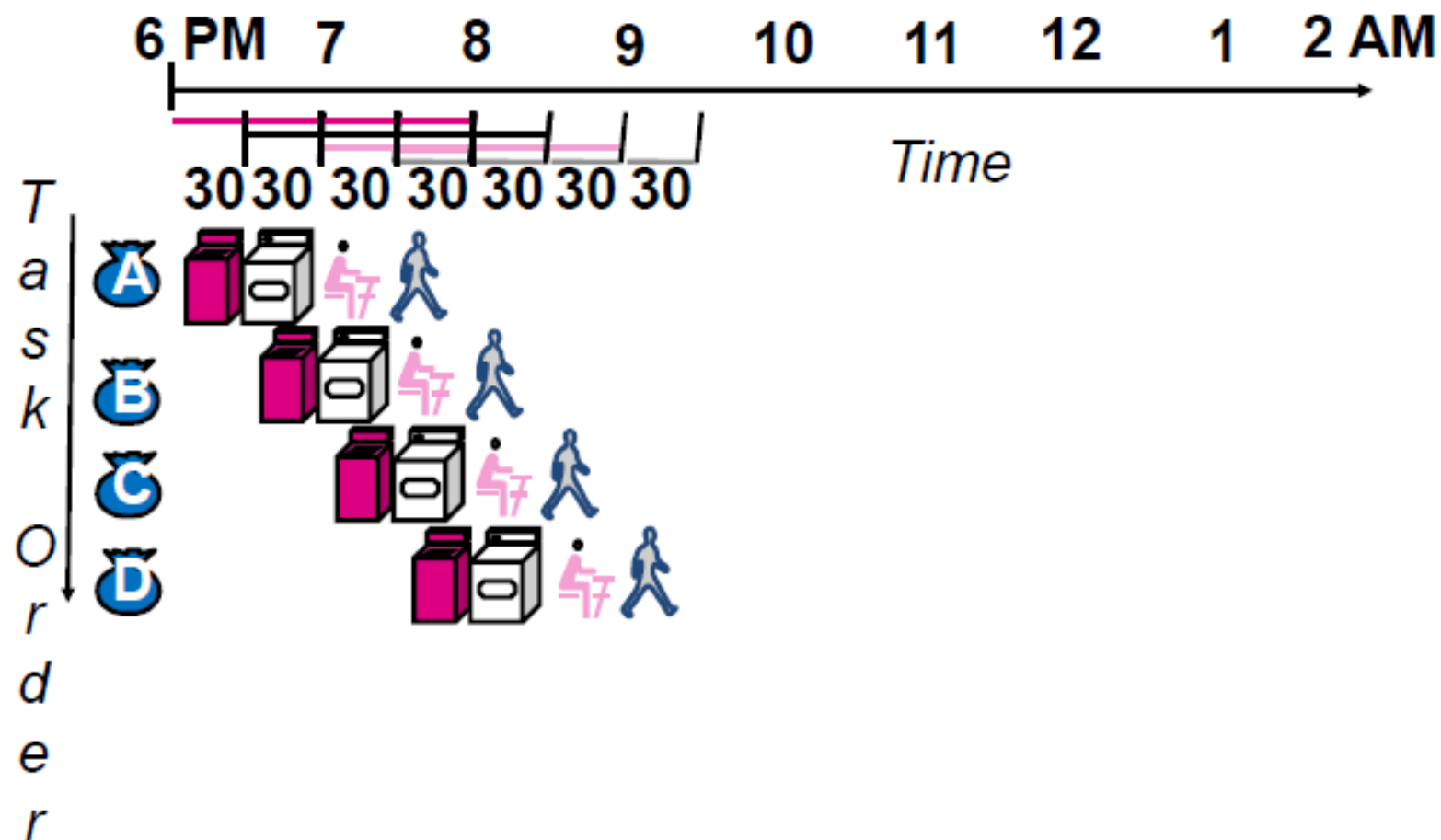


Pipelined Analogy: Doing Laundry



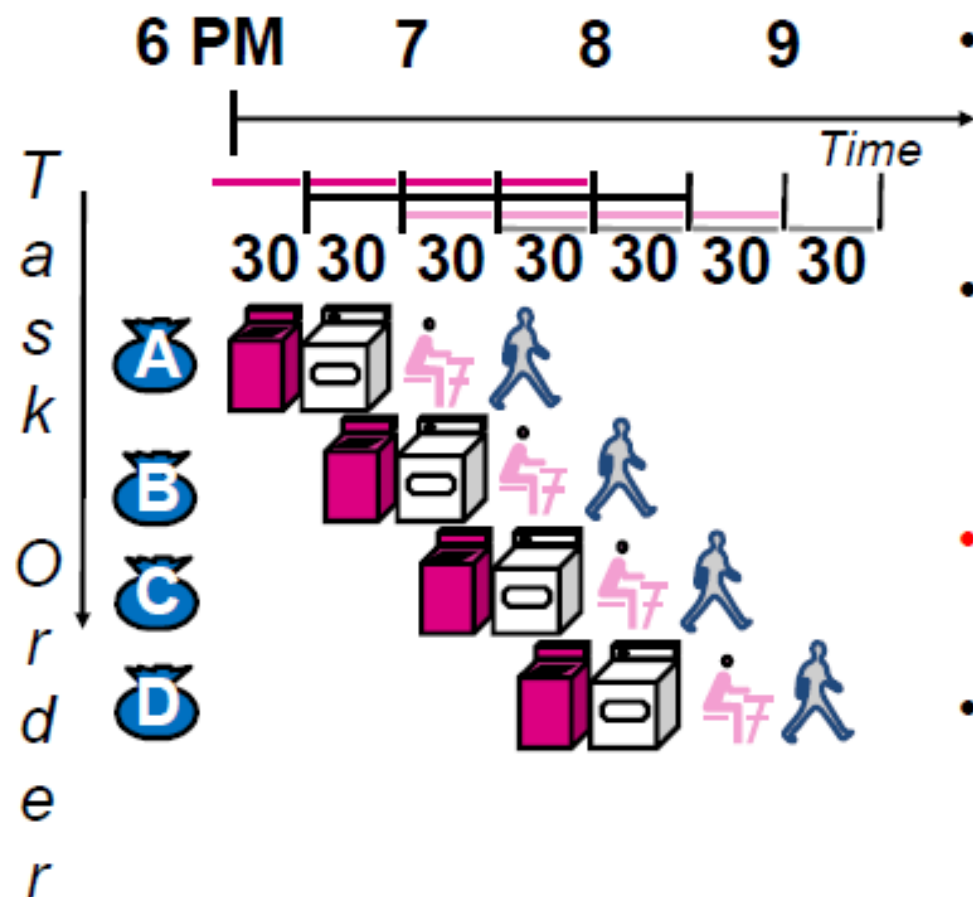
- Sequential laundry takes 8 hours for 4 loads

Pipelined Analogy: Doing Laundry



- *Pipelined laundry takes 3.5 hours for 4 loads!*

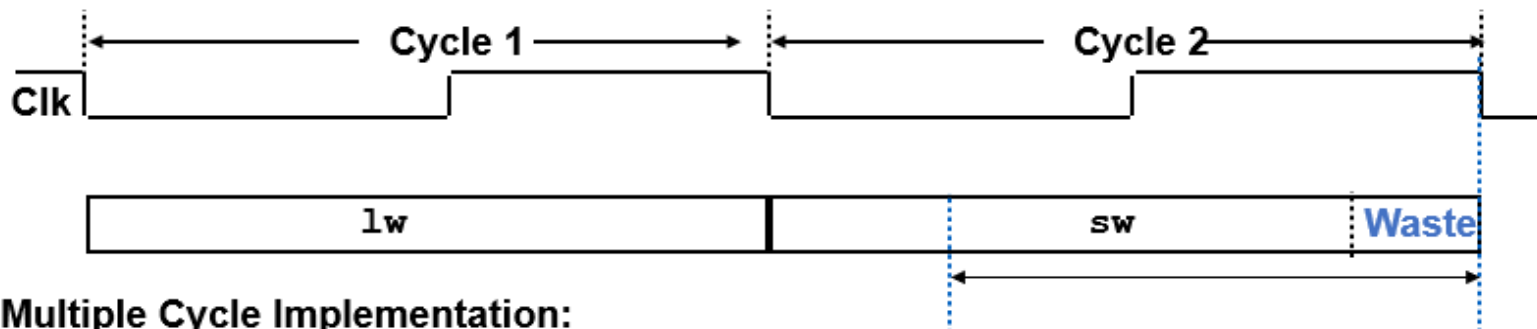
Pipelined Analogy: Doing Laundry



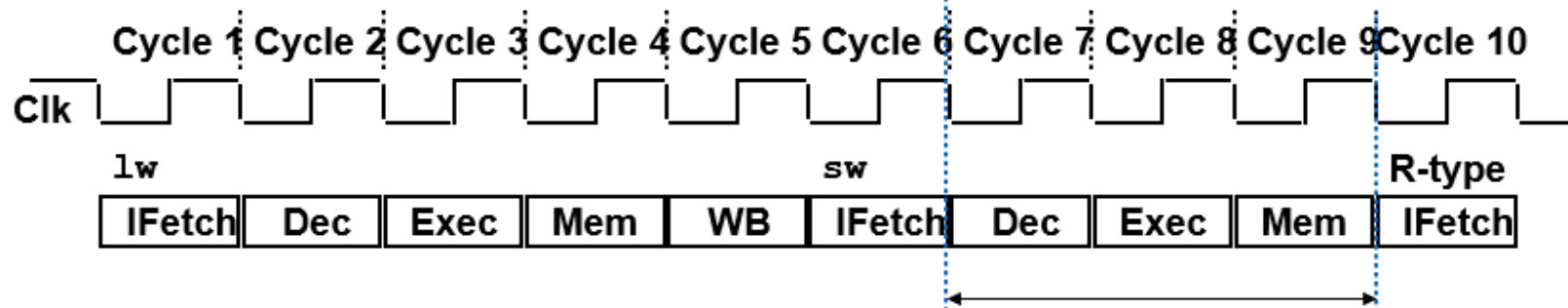
- Pipelining doesn't help *latency* of single task, just *throughput* of entire workload
- *Multiple* tasks operating simultaneously using different resources
- Potential speedup = number of pipeline stages
- Speedup reduced by time to *fill* and *drain* the pipeline:
8 hours/3.5 hours or 2.3X
v. potential 4X in this example

Single Cycle, Multicycle, vs. Pipeline

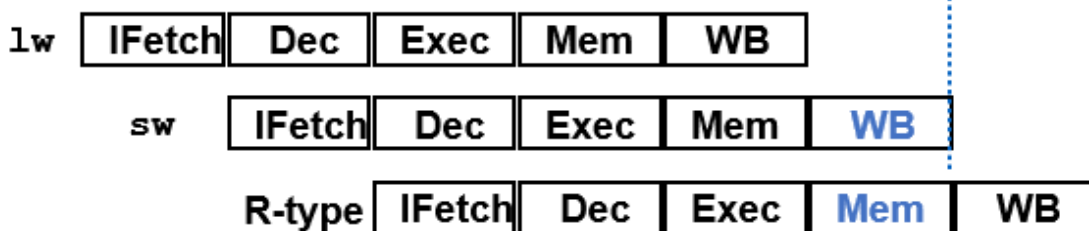
Single Cycle Implementation:



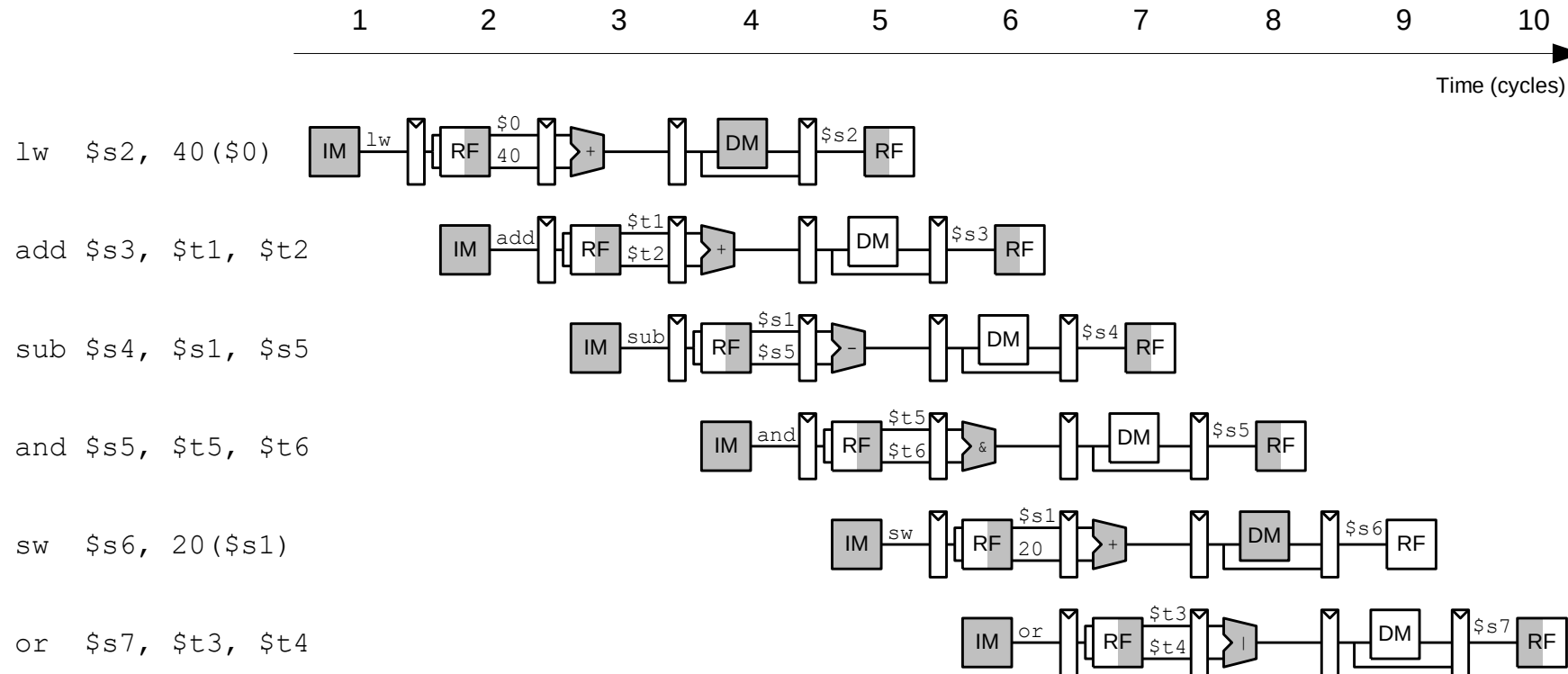
Multiple Cycle Implementation:



Pipeline Implementation:



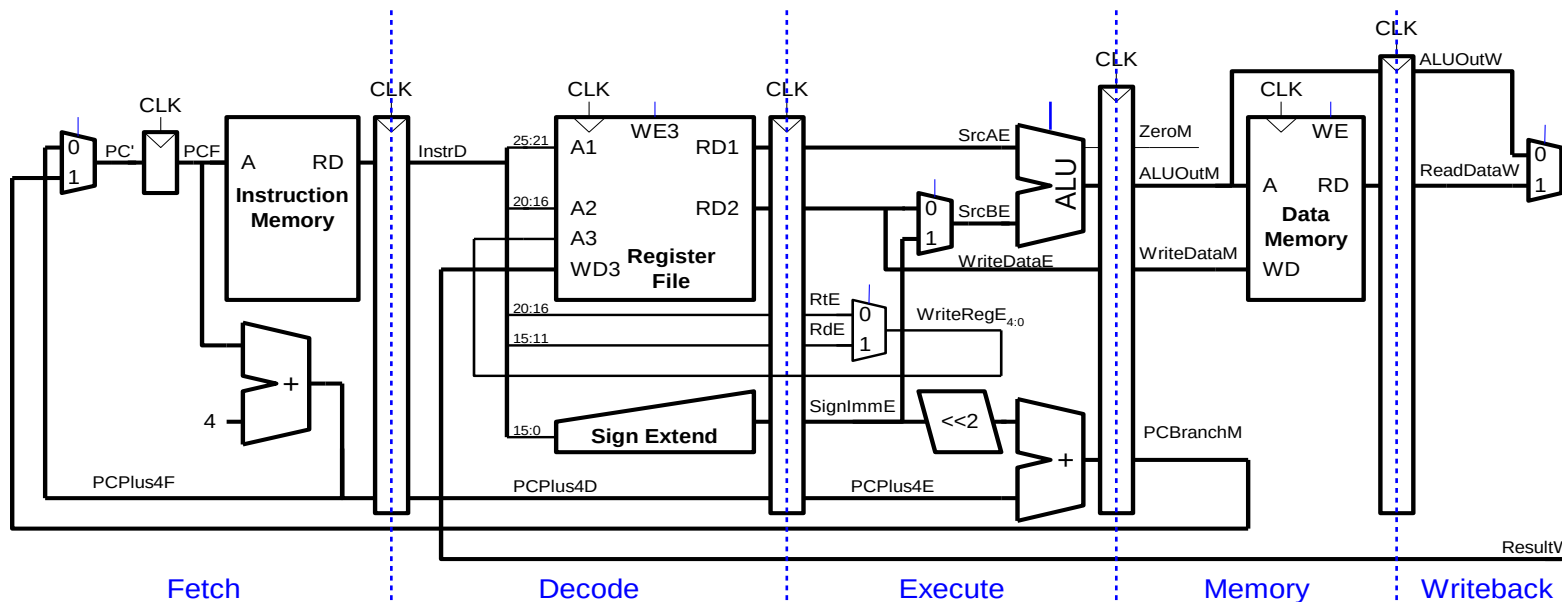
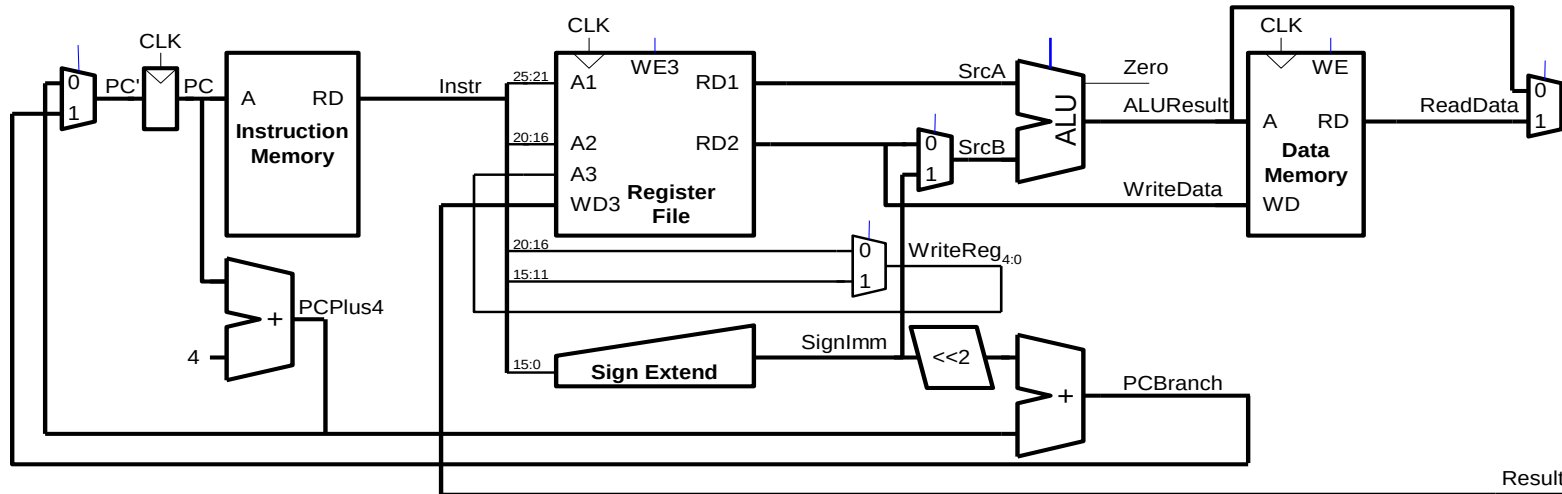
Pipelined Processor



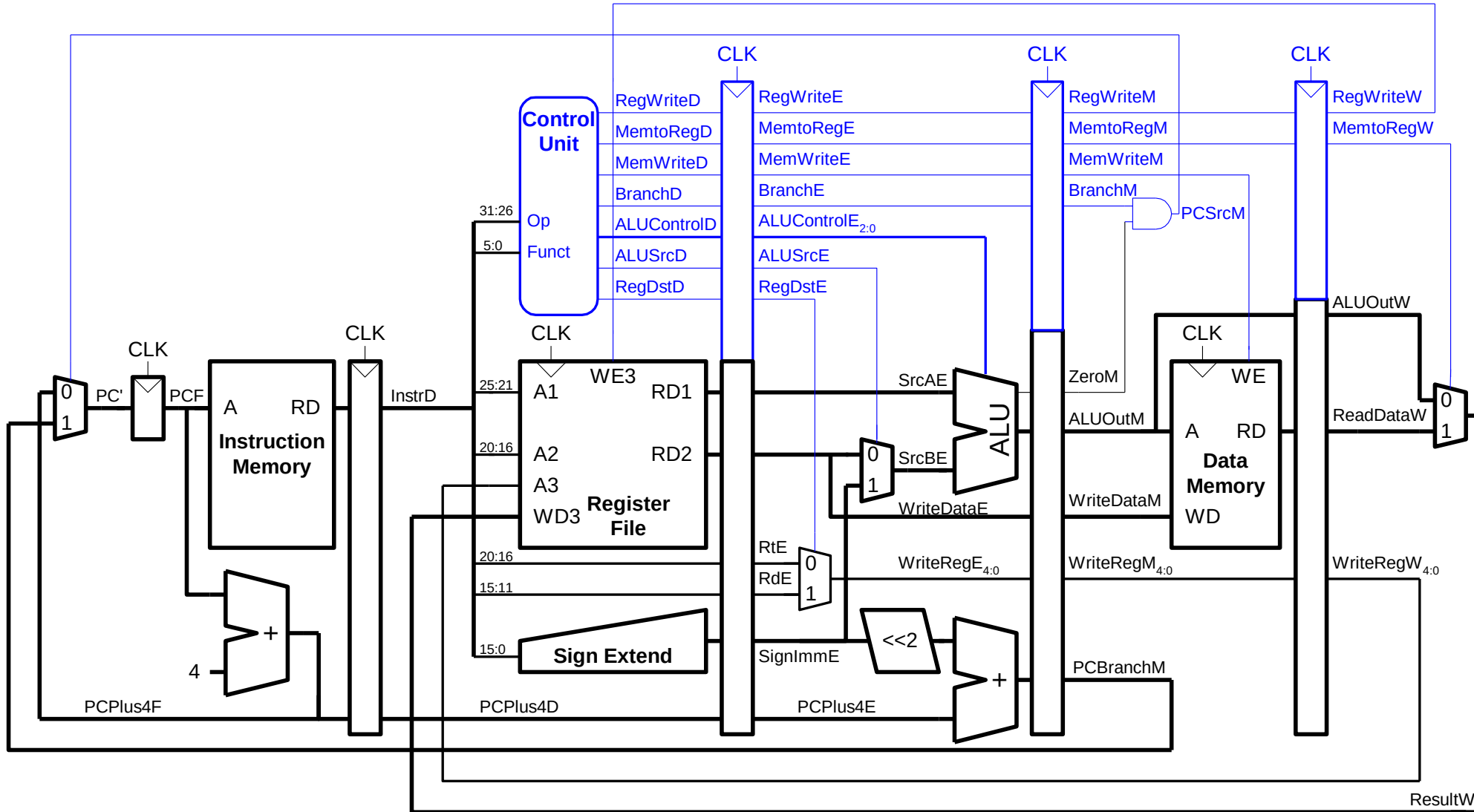
Resource usage

- A hardware resource is designed to be accessed by one instruction at a time

Single-Cycle vs. Pipelined Datapath



Pipelined Processor Control



Pipeline Performance

- Use T_c (“time between completion of instructions”) to measure speedup

$$T_{c,\text{pipelined}} \geq \frac{T_{c,\text{single-cycle}}}{\text{Number of stages}}$$

– Equality only achieved if stages are *balance* (i.e. take the same amount of time)

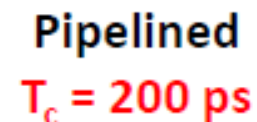
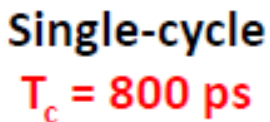
- If not balanced, speedup is reduced
 - Speedup due to increased *throughput*
 - *Latency* for each instruction does not decrease

Pipeline Performance

- Assume time for stages is
 - 100ps for register read or write
 - 200ps for other stages
- What is pipelined clock rate?
 - Compare pipelined datapath with single-cycle datapath

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

Pipeline Performance



$$T_c \text{ Pipeline,} = 200ps > \frac{T_c \text{ Single - cycle}}{5 \text{ stages}} = 160ps$$

Pipeline Hazards

- Problem for pipeline:
 - instruction depends on result from instruction that hasn't completed
- Types:
 - **Structural hazard:** attempt to use the same resource (hardware unit) two different ways at the same time
 - E.g., two instructions try to read the same memory at the same time.
 - **Data hazard:** register value not yet written back to register file
 - E.g., add `$r1, $r2, $r3`
sub `$r4, $r2, $r1`
 - **Control hazard:** next instruction not decided yet (caused by branches)
 - E.g., beq `$r1, $r2, loop`
add `$r3, $r4, $r5`
- Pipeline control must detect hazard
 - Resolve hazards by **waiting / delay action**

CH 6.3:

Memory System and Hierarchy

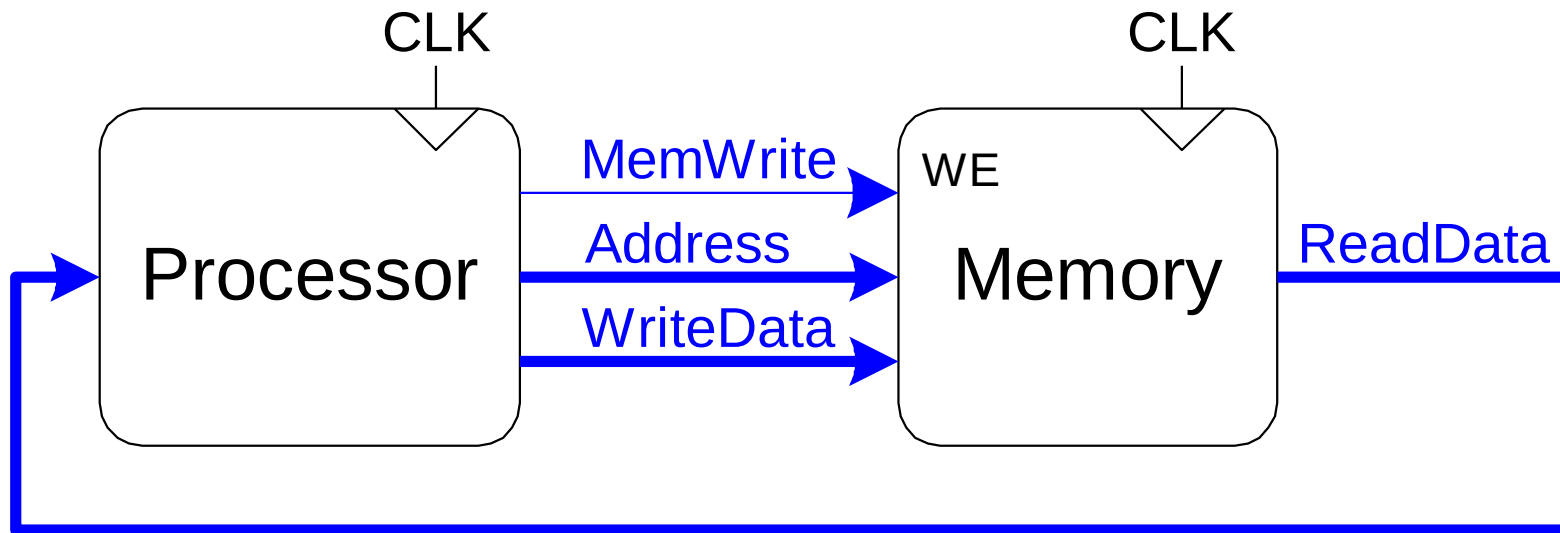


- Memory System Challenge
- Memory Hierarchy
- Memory Type
- Locality

Introduction

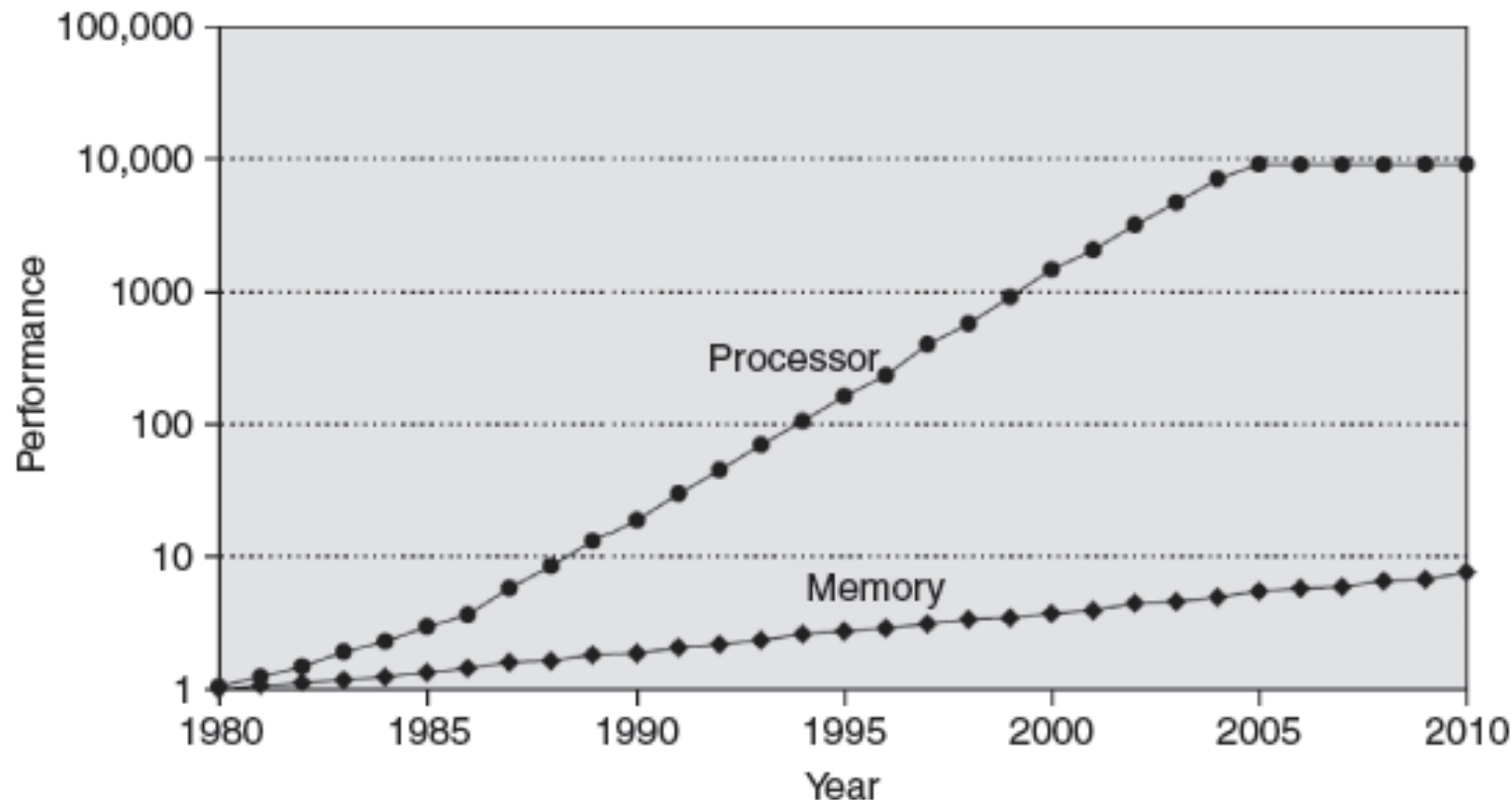
- Computer performance depends on:
 - Processor performance
 - Memory system performance

Memory Interface



Processor-Memory Gap

- In prior chapters, assumed access memory in 1 clock cycle – but hasn't been true since the 1980's



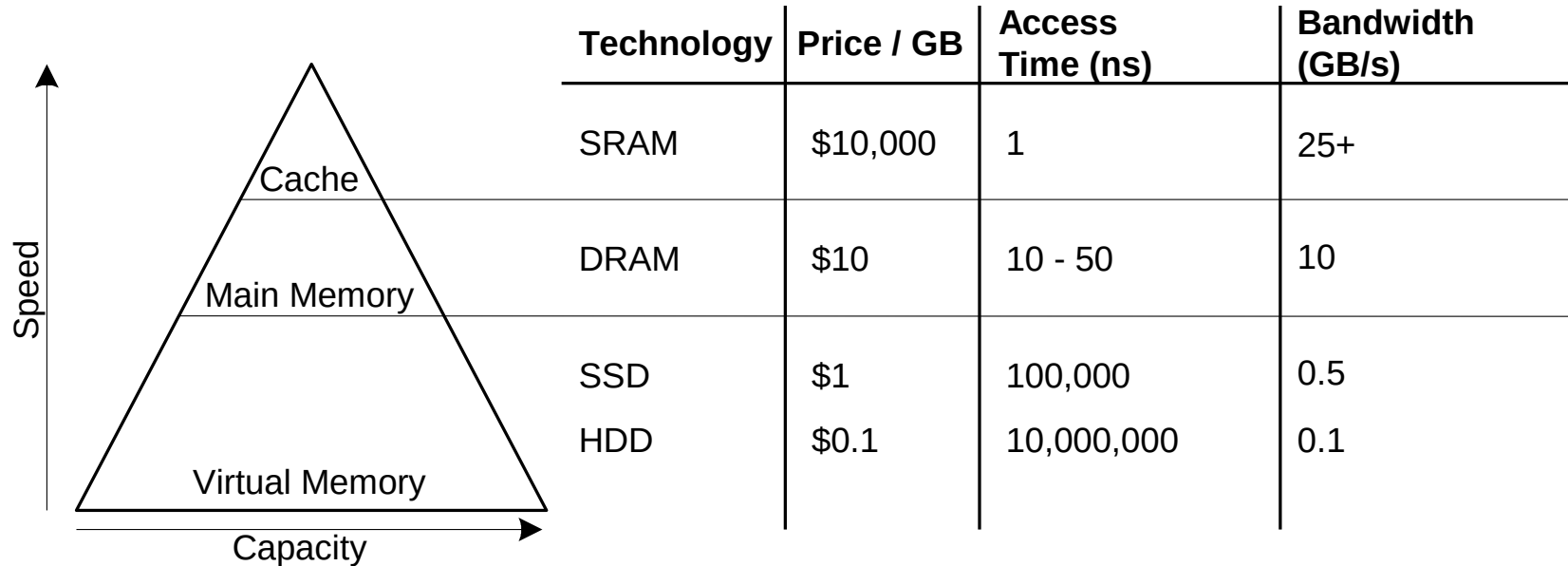
CPU processing speed increases at a much faster rate than the memory access speed

Memory System Challenge

- Make memory system appear as fast as processor
- Use hierarchy of memories
- Ideal memory:
 - Fast
 - Cheap (inexpensive)
 - Large (capacity)

But can only choose two!

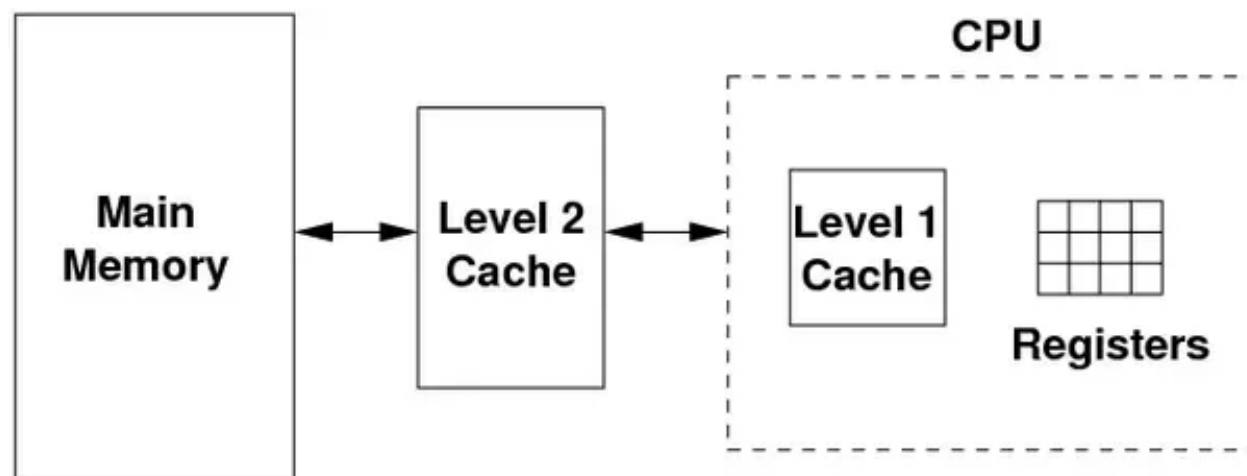
Memory Hierarchy



- **Cache:** SRAM
- **Physical Memory:** DRAM (Main Memory)
- **Virtual Memory:** Hard drive
 - Slow, Large, Cheap

Cache

- Highest level in memory hierarchy
- Fast (typically ~ 1 cycle access time)
- Ideally supplies most data to processor
- Usually holds most recently accessed data

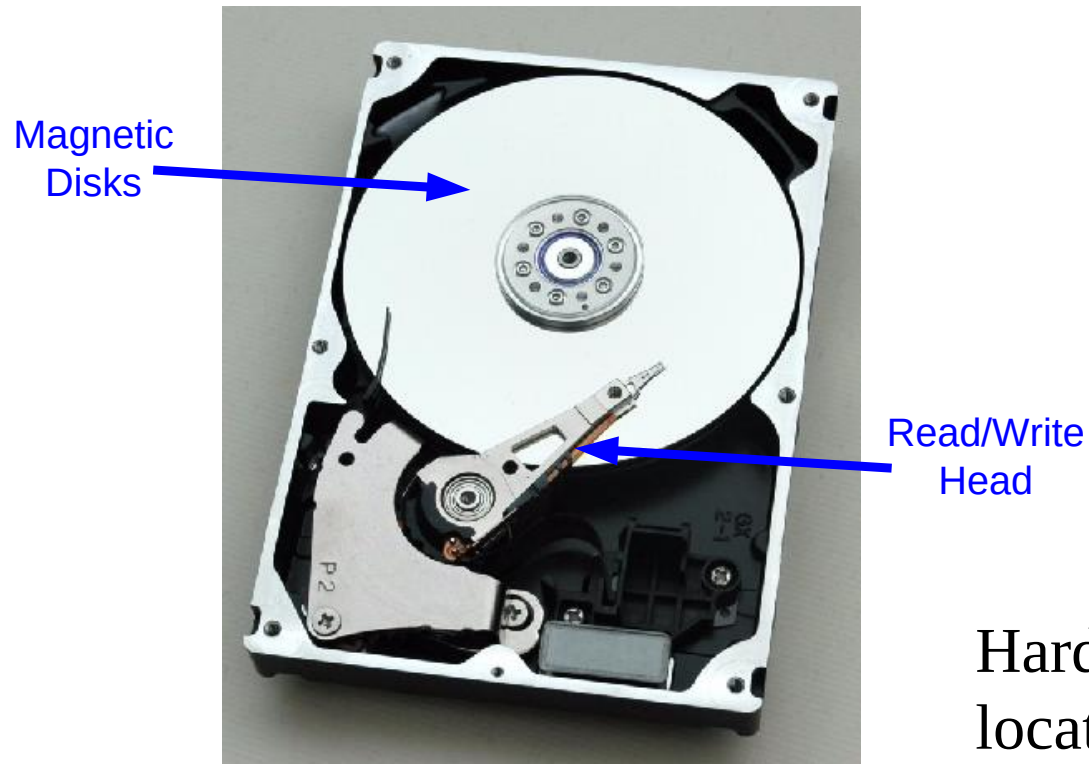


Memory Type

- Types of memory:
 - Random access memory (RAM)
 - Read only memory (ROM)
- RAM is **volatile** – loses its data when power off.
 - Historically called **random access memory** because any data word accessed as easily as any other (in contrast to sequential access memories such as a tape recorder)
 - The main memory in your computer is **dynamic random access memory** (DRAM).
 - DRAM stores data using capacitor
- ROM is **nonvolatile** – it retains data when power off
 - Flash memory in cameras, thumb drives, and digital cameras are all ROMs.
 - Historically called **read only memory** because ROMs were written at manufacturing time or by burning fuses. Once ROM was configured, it could not be written again. This is no longer the case for Flash memory and other types of ROMs.

Virtual Memory

- Gives the illusion of bigger memory
- Main memory (DRAM) acts as cache for hard disk



Hard disk takes milliseconds to seek the correct location on disk

Principle of Locality

Exploit locality to make memory accesses fast

- **Temporal Locality:**

- Locality in time
- If data used recently, likely to use it again soon
- **How to exploit:** keep recently accessed data in higher levels of memory hierarchy
- E.g., instructions in a loop.

- **Spatial Locality:**

- Locality in space
- If data used recently, likely to use nearby data soon
- **How to exploit:** when access data, bring nearby data into higher levels of memory hierarchy too
- E.g., Sequential instruction access, array data.

Taking Advantage of Locality

- Store everything on disk
- Copy recently accessed (and nearby) items from disk to smaller DRAM memory
 - Main memory
- Copy more recently accessed (and nearby) items from DRAM to smaller SRAM memory
 - Cache memory attached to CPU



Q & A
Thank you